

Gaussian Processes for UUV Positioning

gp-research knowledge base - references + implementation

gp-research/ — Gaussian Processes for UUV Positioning

A self-contained knowledge base for using **Gaussian Processes (GPs)** to model the position of an underwater unmanned vehicle (UUV/AUV) in this repository's navigation stack.

It is split in two:

- **`references/`** — teaching material. Concepts, math, intuition, and small worked examples.

Written to be generally educational; read these to understand before building.

- **`implementation/`** — build detail specific to this codebase (the DVL/IMU error-state EKF in

dvL_correction/). Read these when you are ready to design and code.

Scope note: this round is documentation only. No source code, config, or dependencies are

changed. Implementation happens in a later, separately-approved round, using implementation/00-implementation-plan.md as the spec.

Why GPs for an underwater vehicle?

Underwater there is **no GPS**. Position comes from dead-reckoning: integrate IMU acceleration and

DVL (Doppler Velocity Log) velocity over time. Integration means **error grows without bound**, and

the DVL — our best velocity sensor — **drops out** (loss of bottom-lock over soft/steep terrain, aeration, steep banking) and occasionally returns **outliers**. A Gaussian Process is a natural fit because it:

1. Models a quantity (velocity / position) **as a function of time or space**, with calibrated uncertainty at every query point — exactly what a Kalman filter wants as a measurement.

2. **Interpolates and extrapolates** smoothly, so it can **bridge DVL gaps** with an uncertainty

that grows the longer the gap — graceful degradation, not a cliff.

3. Can be made to **respect physics** (ocean-current advection-diffusion) and boundary conditions through the SPDE / second-order-PDE duality.

4. Has a **cheap, exact $O(n)$ form** (the state-space / Kalman duality) suitable for the vehicle's

edge compute.

The three themes (and where they live)

Theme	What it answers	Primary docs
GPs for AUV navigation	How does a GP plug into DVL/IMU fusion and improve position?	`references/04`, `implementation/00-03`
The PDE / SPDE duality	How do we inject physical processes and boundary conditions?	`references/06`, `references/07`
Cheap edge compute	How do we run this on the vehicle without a GPU?	`references/05`, `references/08`, `implementation/04`

Suggested reading order

1. references/00-overview-and-reading-map.md
2. references/01-gaussian-processes-fundamentals.md
3. references/02-kernels-and-covariance-functions.md
4. references/03-gp-regression-and-inference.md
5. references/04-gps-for-navigation-and-sensor-fusion.md
6. references/05-state-space-gps-and-the-kalman-duality.md
7. references/06-spde-approach-the-pde-duality.md
8. references/07-physics-informed-gps-and-boundary-conditions.md
9. references/08-cheap-edge-compute-for-gps.md
10. references/09-glossary-and-annotated-references.md

Then the implementation set:

- implementation/00-implementation-plan.md
- implementation/01-codebase-architecture-and-seams.md
- implementation/02-gp-velocity-module-design.md
- implementation/03-benchmark-and-frontend-demo.md
- implementation/04-edge-and-spde-roadmap.md

How references/ maps to implementation/

references/01,02,03	(what a GP is)	—————→	implementation/02 (gp_velocity.py design)
references/04	(GP in nav fusion)	—————→	implementation/00,01 (plan + seams)
references/05,08	(state-space, edge)	—————→	implementation/04 (edge roadmap)
references/06,07	(SPDE, physics, BCs)	—————→	implementation/04 (SPDE roadmap)

Decisions already made (drive the implementation docs)

- First build: a **DVL-dropout velocity GP** (denoise + adaptive measurement noise + gap bridging).
- Posture: **demo-first, accuracy over optimization now**, with an **eventual on-vehicle edge** target.
- Tooling: **use a GP library** (GPyTorch) for the demo build; swap to the state-space form for edge.

See references/09 for all sources.

00 — Overview and Reading Map

What you'll learn: what a Gaussian Process is in one paragraph, why it suits underwater positioning, and how the rest of these docs fit together.

Prereqs: none.

Why it matters here: orientation before the detail.

A Gaussian Process in one paragraph

A **Gaussian Process (GP)** is a probability distribution over functions. Instead of fitting a single curve $f(x)$, a GP says: "any finite set of function values $f(x_1), \dots, f(x_n)$ is jointly Gaussian," fully specified by a **mean function** $m(x)$ (often 0) and a **covariance/kernel function** $k(x, x')$ that says how correlated two outputs are based on how close their inputs are. Given some noisy observations, conditioning this joint Gaussian on the data yields a **posterior**: at any new input x^* you get a predicted **mean** and a predicted **variance**. The variance is small near data and grows as you move away — built-in, honest uncertainty. That uncertainty is the property that makes GPs valuable for sensor fusion.

Why this is the right tool underwater

Underwater navigation has three painful facts:

1. **No GPS.** Position is obtained by integrating velocity/acceleration → unbounded drift.
2. **The DVL drops out and lies.** The Doppler Velocity Log is the workhorse velocity sensor, but it loses bottom-lock (soft mud, steep slopes, altitude limits, bubbles) and emits outliers.
3. **Everything is uncertain and asynchronous.** IMU at ~100 Hz, DVL at ~1 Hz, magnetometer sporadically; each with its own noise.

A GP addresses all three: it produces a **continuous, uncertainty-aware** estimate of velocity (or position) over time, so it can **denoise** good DVL, **reject** outliers, and **bridge** gaps with a variance that grows — telling the downstream Kalman filter exactly how much to trust the bridged value. And through the SPDE duality (doc 06) it can be taught **physics** and **boundary conditions**; through the Kalman duality (doc 05) it can run **cheaply on the vehicle** (doc 08).

The three themes the user asked about

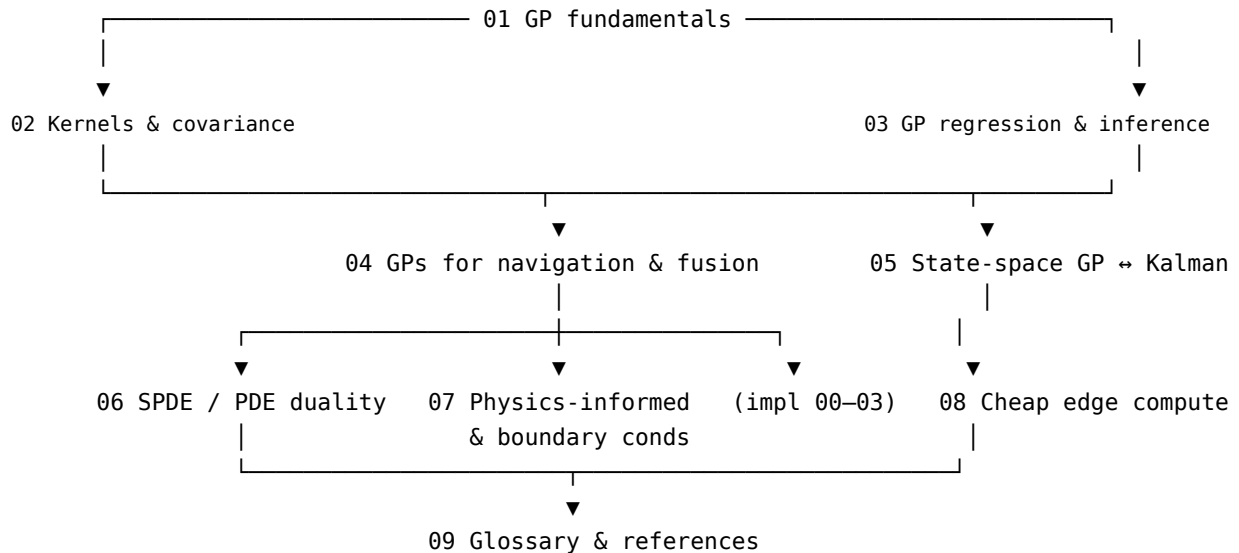
- **GPs for AUV navigation** — the practical "how does it plug in and help" — doc 04, plus all of implementation/.
- **The second-order-PDE / SPDE duality** — "incorporate physical processes and boundary conditions" — docs 06 and 07. This is the deep idea: a Matérn-covariance GP *is the solution of a

stochastic PDE*, so changing the PDE changes the physics the GP assumes.

- **Cheap edge compute** — docs 05 and 08. The key trick: a temporal GP with the right kernel is

mathematically identical to a Kalman filter, which is $O(n)$ and tiny.

Dependency-ordered reading map



- **Just want the build?** Read 01 → 03 → 04, then implementation/.
- **Care most about the PDE duality?** Read 01 → 02 → 06 → 07.
- **Care most about edge compute?** Read 01 → 02 → 05 → 08.

How the math is written here

Equations are plain text/Markdown with backticks, e.g. $k(x, x') = \sigma^2 \exp(-(x-x')^2/(2\tau^2))$.

Vectors

are bold in prose ("**x**"), matrices are capitals (**K**). Where a symbol matters, it is defined inline and collected in doc 09's glossary.

Key takeaways

- A GP = distribution over functions = mean + kernel; its output is a **mean and a variance**.
- The variance (uncertainty) is why it helps sensor fusion: it tells the Kalman filter how much to

trust each value, including bridged values during DVL dropouts.

- Two dualities unlock the advanced asks: **GP ↔ Kalman** (cheap edge) and **GP ↔ SPDE** (physics + boundary conditions).

Further reading

- Rasmussen & Williams, Gaussian Processes for Machine Learning (free online) — the canonical text.
- Continue to 01 — GP fundamentals.

01 — Gaussian Process Fundamentals

What you'll learn: what a GP is mathematically, how conditioning produces a posterior with a predictive mean and variance, and how hyperparameters are learned.

Prereqs: multivariate Gaussian distributions; basic linear algebra.

Why it matters here: this is the machinery behind every later doc and the `gp_velocity` module.

1. From a single function to a distribution over functions

Ordinary regression fits one function $f(x)$ to data. A GP instead maintains a *distribution over all

plausible functions* and updates it with data. Formally:

$$f(x) \sim \text{GP}(m(x), k(x, x'))$$

- $m(x)$ — the **mean function**, our prior guess for the output before seeing data. Usually $m \equiv 0$

after centering.

- $k(x, x')$ — the **covariance (kernel) function**, the heart of the GP. It returns how strongly

$f(x)$ and $f(x')$ covary. Typically: inputs close together $\rightarrow$ outputs strongly correlated; far apart $\rightarrow$ nearly independent.

The defining property: for **any** finite set of inputs $X = \{x_1, \dots, x_n\}$, the vector of outputs $f = [f(x_1), \dots, f(x_n)]^T$ is **jointly Gaussian**:

$$f \sim N(m, K), \quad \text{where } K[i, j] = k(x_i, x_j)$$

K is the $n \times n$ **covariance (Gram) matrix**. Everything a GP does is multivariate-Gaussian algebra on K .

2. Sampling from the prior (intuition)

Before any data, you can draw functions from the GP: pick a dense grid of inputs, build K , and sample $f \sim N(0, K)$. With a smooth kernel you get smooth wiggly curves; with a rough kernel you get

jagged ones. The kernel's **lengthscale** controls how fast the curves wiggle, its **variance** controls their amplitude. (See doc 02 for the menu.)

This is the prior: "here are the kinds of functions I believe in." Data then narrows it down.

3. Conditioning on data $\rightarrow$ the posterior (the key step)

Suppose we observe noisy data $y = f(X) + \varepsilon$, with $\varepsilon \sim N(0, \sigma_n^2 I)$ (i.i.d. observation noise of

variance σ_n^2). We want predictions at new test inputs X^* . Stack training and test outputs; they are jointly Gaussian:

$$\begin{bmatrix} y \\ f^* \end{bmatrix} \sim N \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} K(X, X) + \sigma_n^2 I & K(X, X^*) \\ K(X^*, X) & K(X^*, X^*) \end{bmatrix} \right)$$

Applying the standard Gaussian conditioning formula gives the **posterior** $f^* \mid X, y, X^*$, which is again Gaussian with:

$$\begin{aligned}\text{mean}^* &= K(X^*, X) [K(X, X) + \sigma_n^2 I]^{-1} y \\ \text{cov}^* &= K(X^*, X^*) - K(X^*, X) [K(X, X) + \sigma_n^2 I]^{-1} K(X, X^*)\end{aligned}$$

Read these two lines carefully — they are the whole of GP prediction:

- **Predictive mean** is a weighted average of the training targets y , where the weights come from

how similar (per the kernel) each test point is to the training points.

- **Predictive covariance** starts at the prior $K(X^*, X^*)$ and is reduced by a term that grows with how much the data constrains the test point. Near data $\rightarrow$ big reduction $\rightarrow$ small variance. Far

from data $\rightarrow$ little reduction $\rightarrow$ variance returns to the prior. This growth-away-from-data is exactly the property we exploit to bridge DVL dropouts (doc 04).

Let $\alpha = [K + \sigma_n^2 I]^{-1} y$ (computed once). Then for a single test point x^* :

$$\begin{aligned}\text{mean}(x^*) &= k(x^*, X) \cdot \alpha \\ \text{var}(x^*) &= k(x^*, x^*) - k(x^*, X) [K + \sigma_n^2 I]^{-1} k(X, x^*)\end{aligned}$$

4. A tiny worked example (1-D, by hand intuition)

Say we use the squared-exponential kernel $k(x, x') = \exp(-(x-x')^2/2)$ (unit variance, unit lengthscale)

and observe two noise-free points: $f(0)=0$, $f(2)=1$.

- At $x^*=0$ (on a data point): the posterior mean is 0 and variance ≈ 0 — the GP is certain.
- At $x^*=1$ (between the two points): the mean interpolates to roughly 0.5, with **small** variance

— it is "boxed in" by neighbours on both sides.

- At $x^*=5$ (far to the right of all data): the mean decays back toward the prior mean 0, and the

variance climbs back toward the prior variance 1 — the GP admits "I don't know out here."

That qualitative behaviour — confident interpolation, honest extrapolation — is the entire reason GPs

are useful for filling sensor gaps.

5. Hyperparameters and the marginal likelihood

The kernel has free **hyperparameters** θ (lengthscale, variance) plus the noise σ_n^2 . We don't guess them; we learn them by maximizing the **log marginal likelihood** of the observed data:

$$\log p(y \mid X, \theta) = -\frac{1}{2} y^T (K\theta + \sigma_n^2 I)^{-1} y - \frac{1}{2} \log |K\theta + \sigma_n^2 I| - (n/2) \log 2\pi$$

- The first term rewards **fitting the data** (small when y is explained well).
- The second term, $-\frac{1}{2} \log |K + \sigma_n^2 I|$, is a **complexity penalty** (Occam's razor) — it punishes overly

flexible kernels. This automatic trade-off is one of the GP's most attractive features: it resists overfitting without a separate validation set.

In practice we maximize this with gradient ascent (e.g. Adam). Libraries like GPyTorch

compute the
gradients by autodiff — see `implementation/02`.

6. What you get out, and why it's special

A trained GP returns, for any query input, `(mean, variance)` — a prediction *with calibrated uncertainty*. Most regressors give only a point estimate. The variance is what a Kalman filter needs

as its measurement-noise term R , which is precisely how we will couple the GP to the existing EKF

(doc 04,
`implementation/00`).

7. The catch (motivates docs 03, 05, 08)

The posterior requires solving $[K + \sigma_n^2 I]^{-1}$ for an $n \times n$ matrix — $O(n^3)$ to factorize and $O(n^2)$ memory. With n = thousands of DVL samples this is the central scalability problem. Three

escapes: keep n small (sliding window), use the **state-space/Kalman** form for $O(n)$

(doc 05), or use **sparse/inducing-point**

approximations (doc 08).

Key takeaways

- GP = mean function + kernel; any finite set of outputs is jointly Gaussian.
- Conditioning on data gives a Gaussian **posterior**: predictive **mean** (data-weighted) and **variance** (small near data, grows away — great for gap-filling).
- Hyperparameters are learned by maximizing the **marginal likelihood**, which auto-balances fit vs complexity.
- The cost is $O(n^3)$; later docs show how to dodge it.

Further reading

- Rasmussen & Williams, GPML, ch. 2 (regression) — the source for all formulas above.
- Next: 02 — Kernels and covariance functions.

02 — Kernels and Covariance Functions

What you'll learn: the common kernels (RBF, Matérn, Rational Quadratic, periodic), what their hyperparameters do, how to combine kernels, and multi-output/multitask kernels for correlated x/y/z velocity.

Prereqs: 01 — GP fundamentals.

Why it matters here: the kernel encodes every assumption the GP makes; our velocity model uses a specific composite, multitask kernel.

1. What a kernel is and must be

The kernel $k(x, x')$ defines the covariance between outputs at two inputs. It controls smoothness, amplitude, and how far correlations reach. To be valid, a kernel must be **positive semi-definite** (every Gram matrix K it produces must be PSD) — this is what guarantees the joint distribution is a legal Gaussian. The good news: you rarely build kernels from scratch; you compose known-valid ones.

Two near-universal hyperparameters:

- **Variance / signal amplitude** σ^2 — vertical scale of the function (how big the wiggles are).
- **Lengthscale** ℓ — horizontal scale (how far apart inputs must be before outputs decorrelate).

Small ℓ $\rightarrow$ wiggly, fast-changing; large ℓ $\rightarrow$ smooth, slowly-changing. This is usually the most important knob.

2. The squared-exponential (RBF) kernel

$$k_{SE}(x, x') = \sigma^2 \cdot \exp\left(-\frac{(x-x')^2}{2\ell^2}\right)$$

The default smooth kernel. Samples are **infinitely differentiable** — extremely smooth. That is often too smooth for physical signals (real motion is not infinitely smooth), which is why Matérn is usually preferred for navigation.

3. The Matérn family (the workhorse for physical signals)

$$k_{\text{Matérn}, \nu}(r) = \sigma^2 \cdot (2^{\{1-\nu\}}/\Gamma(\nu)) \cdot (\sqrt{2\nu} \, r/\ell)^{\nu} \cdot K_{\nu}(\sqrt{2\nu} \, r/\ell), \quad r = |x-x'|$$

The smoothness parameter ν controls how many times sample paths are differentiable. Three values have closed forms and dominate practice:

- **Matérn-1/2** ($\nu=1/2$): $k = \sigma^2 \exp(-r/\ell)$. This is the **exponential / Ornstein-Uhlenbeck**

kernel — continuous but non-differentiable (rough). Equivalent to a 1st-order Markov process.

- **Matérn-3/2** ($\nu=3/2$): $k = \sigma^2 (1 + \sqrt{3} r/\ell) \exp(-\sqrt{3} r/\ell)$. **Once differentiable.** A very

common, robust default for real-world signals — smooth enough to look physical, rough enough not to

over-smooth. **This is the backbone of our velocity kernel.**

- **Matérn-5/2** ($\nu=5/2$): $k = \sigma^2 (1 + \sqrt{5} r/\ell + 5r^2/3\ell^2) \exp(-\sqrt{5} r/\ell)$. **Twice differentiable.**

Good when you believe the signal has a meaningful acceleration.

As $\nu \rightarrow \infty$, Matérn $\rightarrow$ RBF. **Why this matters for us:** vehicle velocity is a physical quantity with

finite smoothness; Matérn-3/2 or -5/2 models it far better than the unrealistically smooth RBF.

Matérn also has a deep second role — it is the kernel that connects GPs to **SPDEs** and to **state-space/Kalman** forms (docs 05 and 06). Choosing Matérn now keeps both efficiency doors open.

4. Rational Quadratic (RQ)

$$k_{\text{RQ}}(x, x') = \sigma^2 \cdot (1 + (x - x')^2 / (2\alpha^2))^{-\alpha}$$

An infinite mixture of RBF kernels with different lengthscales (parameter α sets the spread). It captures variation happening at **several scales at once** — useful when the signal has both slow trends and faster fluctuations. As $\alpha \rightarrow \infty$ it reduces to RBF.

5. Periodic and other structural kernels

$$k_{\text{per}}(x, x') = \sigma^2 \cdot \exp(-2 \sin^2(\pi |x - x'| / p) / \ell^2)$$

Encodes repetition with period p — e.g. a lawnmower survey pattern or a periodic disturbance. Probably not needed for the first velocity model but worth knowing for richer trajectory models.

6. ARD — automatic relevance determination

When the input is multi-dimensional ($x \in \mathbb{R}^d$), give each dimension **its own lengthscale**:

$$k(x, x') = \sigma^2 \cdot \exp(-\frac{1}{2} \sum_d (x_d - x'_d)^2 / \ell_d^2)$$

If dimension d is irrelevant, training drives $\ell_d \rightarrow \infty$ and that input is effectively ignored — the model learns which inputs matter. Relevant if we later feed the GP extra features (attitude, specific force) alongside time.

7. Combining kernels (the composable algebra)

Valid kernels are closed under **addition** and **multiplication**:

- **Sum** $k_1 + k_2$: the function is a sum of independent components (e.g. a smooth trend **plus**

multi-scale fluctuation). Intuitively "OR".

- **Product** $k_1 \cdot k_2$: components must both be active (e.g. periodic **and** slowly decaying).

Intuitively "AND".

Our velocity kernel is a sum Matérn-3/2 + RQ + RBF over time: Matérn-3/2 supplies physically plausible local smoothness, RQ captures multi-scale variation, RBF a smooth long trend. This mirrors the composite-ARD choice reported in the underwater-navigation GP literature (Cohen & Klein 2025, see doc 09). It is intentionally expressive because this round prioritizes accuracy over compute.

8. Multi-output / multitask kernels (correlated x, y, z)

The DVL gives a **3-vector** velocity (v_x , v_y , v_z) in the body frame, and the axes are correlated (the vehicle moves as one rigid body). Modelling them as three independent GPs throws that away.

Multitask / multi-output GPs model all outputs jointly.

The standard construction is the **Linear Model of Coregionalization (LMC)** and its special case the

Intrinsic Coregionalization Model (ICM):

$$k((x, i), (x', j)) = B[i, j] \cdot k_{\text{time}}(x, x')$$

- k_{time} is the shared temporal kernel from §7.
- B is a small (here 3×3) PSD **coregionalization matrix** of learned inter-axis correlations.

So "how correlated is v_x at time t with v_z at time t' " = $B[x, z] \times$ (temporal similarity). ICM uses

one shared k_{time} ; LMC sums several such terms with different kernels for richer cross-axis structure. GPyTorch implements these as MultitaskKernel (ICM) / LCMKernel (LMC). Our model uses a

multitask kernel so the GP exploits the rigid-body coupling between velocity axes — the same idea

that lets multi-output GP regression beat per-axis fits in the published results.

9. Picking and reading hyperparameters

After training, the learned values are interpretable diagnostics:

- **Lengthscale** $\approx$ the timescale over which velocity stays correlated. If it learns "0.5 s", the GP

can only safely bridge gaps much shorter than that before its variance balloons — directly informs

the dropout-bridge thresholds in `implementation/02`.

- **Signal variance** $\approx$ how much the velocity actually varies — sanity-check against physics.
- **Noise** $\sigma_n^2 \approx$ the DVL's effective measurement noise — should be in the ballpark of the sensor

`spec / configured default_dvl_velocity_std_m_s`.

Key takeaways

- The kernel encodes all assumptions: smoothness (Matérn ν), amplitude (σ^2), reach (ℓ),

multi-scale

(RQ), periodicity, per-input relevance (ARD).

- **Matérn-3/2** is our smoothness backbone — physically realistic and the gateway to the SPDE and

Kalman dualities.

- Kernels **add (OR)** and **multiply (AND)**; our model is a **sum** of Matérn-3/2 + RQ + RBF.
- **Multitask (LMC/ICM)** kernels model correlated (v_x, v_y, v_z) jointly via a learned 3×3 matrix —

better than three independent GPs.

Further reading

- Rasmussen & Williams, GPML, ch. 4 (covariance functions).
- Duvenaud, The Kernel Cookbook (online) — excellent intuition for composing kernels.
- Álvarez, Rosasco, Lawrence, Kernels for Vector-Valued Functions — multi-output GPs.
- Next: 03 — GP regression and inference.

03 — GP Regression and Inference

What you'll learn: the exact GP regression algorithm step by step, the Cholesky-based computation, where the $O(n^3)$ cost comes from, and practical issues (noise, jitter, predictive covariance).

Prereqs: 01, 02.

Why it matters here: this is literally what `GpDvlVelocityModel.fit() / .predict()` will run, and the cost analysis is why edge needs docs 05/08.

1. The exact regression algorithm

Given training inputs X , targets y , noise σ_n^2 , kernel k , and test inputs X^* :

1. Build the training covariance $K = K(X, X)$ ($n \times n$) and add noise: $K_y = K + \sigma_n^2 I$.
2. **Cholesky factorize** $K_y = L L^T$ (L lower-triangular). This is the expensive step.
3. Solve for α : $\alpha = L^T \setminus (L \setminus y)$ (two triangular solves). α encodes the fitted weights.
4. Predictive mean: $\text{mean}^* = K(X^*, X) \cdot \alpha$.
5. Predictive covariance: solve $v = L \setminus K(X, X^*)$, then $\text{cov}^* = K(X^*, X^*) - v^T v$.
6. (Training) Log marginal likelihood: $-\frac{1}{2} y^T \alpha - \sum \log \text{diag}(L) - (n/2) \log 2\pi$.

Steps 4-5 are the two outputs we care about; step 6 is what hyperparameter optimization maximizes.

2. Why Cholesky (not a raw inverse)

We never form K_y^{-1} explicitly. Cholesky + triangular solves are faster and far more numerically

stable than computing an inverse. The factor L is reused for the mean, the covariance, and the marginal likelihood (its log-determinant is just $2 \sum \log \text{diag}(L)$). This is the standard, robust recipe; GPyTorch does the equivalent (with extra tricks) under the hood.

3. The cost — and why it dominates everything

- **Time:** the Cholesky factorization of an $n \times n$ matrix is $O(n^3)$.
- **Memory:** storing K is $O(n^2)$.

At $n = 1,000$ that's $\sim 10^9$ flops per fit and ~ 8 MB; at $n = 10,000$ it's $\sim 10^{12}$ flops and ~ 800 MB —

infeasible to refit at sensor rate, and hopeless on an embedded vehicle CPU. This single fact drives the entire architecture:

- For the demo we keep n small with a **sliding window** of recent DVL samples (e.g. last 60-180 s

at ~ 1 Hz $\rightarrow n \approx 60-180$), so $O(n^3)$ is trivial.

- For edge we switch to the **state-space/Kalman** form, which is $O(n)$ exact for temporal data

(doc 05), or to **sparse/inducing-point** approximations (doc 08).

4. Observation noise σ_n^2 and heteroscedastic noise

The $\sigma_n^2 I$ term models sensor noise. Two refinements matter for us:

- **Per-sample noise.** Our DVL measurements can carry their own covariance (the repo's `RawDvlMeasurement.velocity_covariance_dvl / CorrectedDvlMeasurement.covariance_body`). Replace the scalar $\sigma_n^2 I$ with a **diagonal** `Σ_noise`

built from per-sample variances so noisy samples are trusted less. This is standard and supported

by GP libraries.

- **Heteroscedasticity.** If noise varies with operating condition (altitude, beam count), the noise itself can be modelled — out of scope for the first build but noted.

5. Jitter / numerical conditioning

If two inputs are nearly identical or the kernel is very smooth, K_y becomes near-singular and Cholesky fails. The fix is **jitter**: add a tiny ϵI (e.g. $1e-6 \sigma^2$) to the diagonal. GPyTorch adds adaptive jitter automatically. Practically: prefer Matérn over RBF (better conditioned), keep

the sliding window from containing duplicate timestamps, and center/scale inputs.

6. The predictive covariance — our payload

Step 5 produces cov^* , the **predictive covariance** at the query point(s). For a single time it is a 3×3 matrix over (v_x, v_y, v_z) (with the multitask kernel of doc 02 §8).

This matrix is the deliverable we hand to the EKF as the **adaptive measurement-noise** `R` (doc 04). Key behaviours to expect:

- On/near a data time $\rightarrow$ small cov^* (denoised, trustworthy) $\rightarrow$ EKF weights it heavily.
- Inside a long dropout $\rightarrow$ cov^* inflates toward the prior $K(X^*, X^*) \rightarrow$ EKF down-weights it, smoothly handing authority back to IMU dead-reckoning. This is the graceful-degradation mechanism, falling straight out of the math — we don't hand-tune it.

7. Input/target preprocessing (small but important)

- **Center time** within the window (subtract the window's start) to keep numbers small and well-conditioned.

- **Standardize targets** (subtract mean, divide by std per axis) so default hyperparameter priors

behave; undo the transform on output.

- **Sort by time** and drop exact-duplicate timestamps (the pipeline already orders events by time,

see `implementation/01`).

8. From batch to streaming

Exact GP regression is a batch operation (refit on the current window). For a stream we either:

- **Refit periodically** on the sliding window (every `refit_every` samples) — simple, used for the

demo; hyperparameters change slowly so we don't refit every sample.

- **Update recursively** — which, for a Markovian kernel, is the Kalman filter

(doc 05). This is the elegant streaming/edge form.

Key takeaways

- Exact GP regression = build K_y , Cholesky L , solve for α , then mean = $K(X^*, X)\alpha$ and covariance = $K(X^*, X^*) - v^T v$.
- Cost is **$O(n^3)$ time / $O(n^2)$ memory** — the central constraint; we control it with a sliding window now and the Kalman form later.
- Per-sample noise lets us pass the DVL's own covariance in; **jitter** keeps Cholesky stable.
- The **predictive covariance** is the product we feed the EKF, and its growth during gaps gives graceful degradation for free.

Further reading

- Rasmussen & Williams, GPML, Algorithm 2.1 (the exact recipe above).
- GPyTorch docs — exact GP regression tutorial.
- Next: 04 — GPs for navigation and sensor fusion.

04 — GPs for Navigation and Sensor Fusion

What you'll learn: how dead-reckoning and the error-state EKF work in brief, the concrete ways a GP improves DVL/IMU fusion, and a walkthrough of the published underwater-GP results and what transfers to our sensor setup.

Prereqs: 01-03.

Why it matters here: this is the bridge from "what is a GP" to "what we actually build."

1. The underwater navigation problem, briefly

With no GPS, an AUV estimates pose by **dead-reckoning**: integrate IMU specific force/angular rate

(at ~100 Hz) for a high-rate but **drifting** estimate, and correct it with **DVL** body-frame velocity (~1 Hz) and a **magnetometer** heading. This repo does exactly that with an error-state

(indirect) Extended Kalman Filter in `dvl_correction/src/dvl_imu_kalman.py`.

Error-state EKF in one paragraph

Rather than filtering the full nonlinear state, the error-state EKF tracks a small **error** between the true state and the IMU-propagated nominal state. IMU samples **propagate** the nominal state and

grow the error covariance; DVL/mag measurements **update** (shrink) it. Our filter's 16-element error

state is `[position(3), velocity(3), attitude(3), gyro-bias(3), accel-bias(3), declination(1)]`. Each

measurement update needs three things: a **residual** (measurement – prediction), a measurement model

H, and a **measurement-noise covariance R**. *The GP's job is to supply better DVL velocity values and, crucially, a better, time-varying R.* (Code anchors in `implementation/01.`)

2. Four ways a GP improves DVL/IMU fusion

- 1. Denoising.** Replace each raw DVL velocity with the GP posterior **mean** over a recent window.

The GP smooths sensor noise using temporal (and cross-axis) correlation, so the EKF ingests a cleaner measurement.

- 2. Adaptive measurement noise R_k .** Feed the GP's predictive **covariance** as the EKF's R for

that update — per measurement, instead of a fixed constant. When the GP is confident, R is small and the filter leans on the DVL; when not (sparse/odd data), R grows and the filter protects itself. **This is the single highest-value idea** and the core of the published gains.

- 3. Outlier rejection.** Score each incoming DVL sample by its **Mahalanobis distance** to the GP's

predictive band; reject the gross ones before they corrupt the filter. (Our EKF already has a Mahalanobis gate; the GP adds a second, smarter pre-filter.)

4. Dropout bridging.

During DVL silence, **predict** velocity from the GP at the expected DVL cadence, with covariance that **grows with the gap length**. The filter keeps getting velocity updates but trusts them less and less — a smooth fallback to IMU-only, rather than a hard switch.

All four use only the GP's two outputs (mean, covariance) from doc 03.

3. Why "predicted covariance as R_k " is principled, not a hack

A Kalman update is optimal **only if R reflects the true measurement uncertainty**. A fixed R is wrong whenever conditions change (which underwater they always do). The GP's predictive variance is a calibrated estimate of how uncertain this particular velocity value is, given the recent data and learned dynamics. Plugging it in makes the filter's uncertainty bookkeeping honest — exactly what the error-state EKF assumes. This is why the approach improves not just velocity but downstream states like orientation: better-weighted velocity updates propagate into a better attitude estimate.

4. What the literature reports

- Cohen & Klein, GPR for Improved Underwater Navigation (2025). **A** multi-output GP regression (MOGPR) **estimates DVL velocity and its covariance, fed into an error-state EKF as velocity measurement + adaptive R_k (loosely-coupled)**. **Reported** $\approx$ 20% velocity-RMSE reduction and up to **42% orientation** improvement on field trajectories. Kernel: a sum of ARD squared-exponential
 - Matérn-3/2 + rational-quadratic, trained by gradient descent (Adam). Honest caveat: MOGPR stores the full training set and inverts a large matrix — **expensive in real time** (motivates our windowing + edge plan).
 - **GP-UKF (Ko & Fox lineage)**. Use a GP to learn the motion/observation model and run it inside an Unscented Kalman Filter. Better tracking and graceful degradation than a fixed parametric model when dynamics are hard to model analytically.
 - **GP for outlier-robust filtering**. GPR regresses "pseudo-measurements" to replace detected outliers in changeable marine environments (robustness under disturbance).

Full citations: doc 09.

5. What transfers to **our** setup (and one key difference)

Aspect | Cohen & Klein 2025 | This repo

GP *inputs* | raw *4-beam* radial velocities | *already-resolved 3-D velocity* (beams are only validity flags)

GP outputs	3-D DVL velocity + covariance	same (3-D body-frame velocity + 3x3 covariance)
Fusion	error-state EKF, adaptive R_k	identical pattern; our EKF already accepts a per-measurement <code>covariance_body</code>
Kernel	ARD-SE + Matérn-3/2 + RQ	same composite, made multitask for cross-axis correlation

The one real difference: **our DVL exposes resolved velocity, not raw beams**. So our GP is a

temporal model time → velocity (denoise/bridge/outlier), rather than a beam→velocity map. This

is simpler and still delivers denoising, adaptive R, outlier rejection, and—uniquely—gap bridging, which the resolved-velocity temporal form is especially good at. If raw beams are exposed

later, the beam→velocity MOGPR can be added as an input-space variant.

6. Loosely- vs tightly-coupled (our choice)

- **Loosely coupled** (our plan): the GP produces a velocity measurement + R; the EKF treats it as

it already treats DVL. Minimal, low-risk, **no EKF-core changes**.

- **Tightly coupled**: fold GP states into the filter (e.g. GP latent force). More powerful, much more

invasive. Out of scope for the first build.

7. Failure modes to respect

- **Garbage in, garbage out**: if the window is full of outliers, the GP learns them. Mitigate with

the EKF gate + GP outlier pre-filter + robust hyperparameter bounds.

- **Over-smoothing**: too-large lengthscale erases real manoeuvres. The learned lengthscale and the

Matérn-3/2 choice guard against this; validate on the benchmark.

- **Over-confident bridging**: if the predictive variance grows too slowly, the filter over-trusts a

long-bridged value. Cap bridging duration (`max_bridge_s`) and verify variance growth on the benchmark (`implementation/03`).

Key takeaways

- A GP improves DVL/IMU fusion four ways: **denoise, adaptive R_k , outlier rejection, gap bridging**.

- **Adaptive R_k** (predicted covariance as measurement noise) is the principled core and the source

of published gains; it even improves orientation downstream.

- Published results (~20–42% gains) use the same EKF + adaptive-R pattern we plan; the only

difference is we model **resolved** velocity over time (our DVL gives no raw beams), which is simpler and excels at **bridging**.

- We use the **loosely-coupled** form: **no EKF-core changes**.

Further reading

- Cohen & Klein 2025 (arXiv:2502.16510); Ko & Fox, GP-BayesFilters; see doc 09.

- Next: 05 — State-space GPs and the Kalman duality.

05 — State-Space GPs and the Kalman Duality

What you'll learn: why a temporal GP with a Markovian kernel is exactly a Kalman filter/smoothing, giving $O(n)$ inference and free interpolation, and how this connects to the repo's imu-drift crate.

Prereqs: 01-03; a passing acquaintance with Kalman filters helps (04 §1).

Why it matters here: this is the route from the demo GP ($O(n^3)$) to an edge GP ($O(n)$), and it reuses physics we already have.

1. The big idea

For **temporal** GPs (input is time) with certain kernels, the $O(n^3)$ cost of doc 03 **vanishes**. The reason: those kernels correspond to a **linear stochastic differential equation (SDE)** driven by white noise. A linear SDE is a **Markov** process — the future depends on the present, not the whole past — and Bayesian inference on a linear-Gaussian Markov process is exactly the **Kalman filter** (forward) and **RTS smoother** (backward). Both are **$O(n)$** in the number of timesteps.

Temporal GP	≡	linear SDE (state-space model)	≡	Kalman filter / RTS smoother
(kernel view)		(dynamics view)		(algorithm view)

Same posterior mean and variance as the batch GP — just computed in linear time by a recursion instead of a cubic matrix factorization.

2. How a kernel becomes a state-space model

You augment the scalar function $f(t)$ into a small **state vector** $s(t) = [f, \dot{f}, \ddot{f}, \dots]$ (the function and a few derivatives), evolving as:

$ds(t) = F s(t) dt + L d\beta(t)$	(linear SDE)
$f(t) = H s(t)$	(we observe the function component)

F, L, H are small constant matrices determined by the kernel; $d\beta$ is white noise. Discretizing over the (possibly irregular) time steps yields the familiar Kalman predict/update matrices.

- **Matérn kernels are exactly representable.** Matérn-1/2 $\rightarrow$ a 1-state SDE (Ornstein-Uhlenbeck);

Matérn-3/2 $\rightarrow$ 2 states; Matérn-5/2 $\rightarrow$ 3 states. The half-integer ν is precisely what makes the spectral density rational and the state finite. This is a second, decisive reason we chose Matérn

in doc 02. (RBF needs an infinite state and is only approximated in state-space form.)

3. Why it's $O(n)$: the precision matrix is block-tridiagonal

The batch GP works with the dense covariance K . The state-space view works with its inverse, the

precision matrix K^{-1} , which for a Markov process is **block-tridiagonal** — each timestep only couples to its immediate neighbours. Solving a block-tridiagonal system is **$O(n)$** , not $O(n^3)$. This is the same structure exploited by Barfoot, Tong & Anderson's *Trajectory Estimation as Exactly Sparse GP Regression* (arXiv:1412.0630): they pose continuous-time robot trajectory estimation as a GP with a "white-noise-on-acceleration" (constant-velocity) prior, note the block-tridiagonal precision, and solve it in linear time.

4. Free, principled interpolation at any time

Because the model is continuous-time, you can query the posterior at **any** timestamp, not just where you have measurements — with closed-form mean and covariance ("GP interpolation"). This is perfect for our asynchronous sensors: query the velocity GP exactly at an IMU step, or at an expected DVL tick during a dropout. The repo already does linear interpolation of IMU between samples (`synchronization.py _interpolate_imu`); the state-space GP generalizes that to a probabilistic, dynamics-aware interpolation with uncertainty.

5. The connection to imu-drift (we already own the prior)

The Rust `imu-drift` crate (`imu-drift/src/processes.rs`) provides closed-form drift means and variances for inertial error processes. These are precisely the marginal statistics of the state-space GP priors:

<code>`imu-drift` process</code>	Variance law	State-space GP equivalent
<code>`IntegratedVelocityRandomWalk`</code>	$\sigma^2(t) = N \cdot t^3/3$	position under a white-noise-on-acceleration (constant-velocity) prior — the Barfoot prior
<code>`VelocityRandomWalk`</code>	$\sigma^2(t) = N \cdot t$	velocity under white-noise-on-acceleration (Wiener)
<code>`GaussMarkov`</code> (OU)	$\sigma^2(t) = \sigma_b^2(1 - e^{-2t/\tau})$	Matérn-1/2 kernel (exponential), 1-state SDE
<code>`ConstantGyroBias`, biases</code>	linear/quadratic mean terms	deterministic drift / random-constant states
<code>`GyroBiasGravityCoupling`</code>	mean $\propto \frac{1}{2}g \cdot b_g \cdot t^3$	cubic cross-coupling term in the augmented state

Implication: the physically-calibrated process-noise parameters in `imu-drift` (from Allan-variance analysis, `imu-drift/src/allan.rs`) can be dropped straight into a state-space GP as its F/L/Q — giving an edge-deployable GP whose prior is grounded in the vehicle's actual IMU characterization, not guessed hyperparameters. This is the heart of the edge roadmap (`implementation/04`).

6. Demo vs edge — same interface

	Demo build (now)	Edge build (later)
Engine	GPyTorch batch GP on a sliding window	state-space GP = Kalman/RTS recursion
Cost	$O(n^3)$ but n is tiny (windowed)	$O(n)$, constant memory
Kernel source	learned hyperparameters	<code>`imu-drift`</code> Allan-variance params
Deps	torch/gpytorch	numpy only (or <code>no_std</code> Rust)

Interface | `ingest()/fit()/predict()` | ****identical****

Because both hide behind the same `predict()/ingest()` API (implementation/02), swapping the engine for edge is an internal change — the pipeline wiring is untouched.

7. Going multi-dimensional / spatio-temporal

The same idea extends to space-time: keep the Markov structure in time (Kalman) and use a spatial basis at each step → **infinite-dimensional Kalman filtering** (Särkkä, Solin, Hartikainen). This is the algorithmic cousin of the SPDE/GMRF approach in doc 06 and the bridge to modelling an ocean-current field efficiently.

Key takeaways

- A temporal GP with a **Matérn** (Markovian) kernel **is** a Kalman filter/RTS smoother → **$O(n)$** inference, same posterior as the batch GP.
- The precision matrix is **block-tridiagonal**; Barfoot et al. use exactly this for continuous-time trajectory estimation.
- You get **closed-form interpolation at any time** — ideal for asynchronous DVL/IMU and for bridging.
- imu-drift's closed-form variances **are** these GP priors; its Allan-variance params can seed an edge-deployable, physically-grounded state-space GP.

Further reading

- Särkkä & Solin, Applied Stochastic Differential Equations (free) — kernels ↔ SDEs.
- Barfoot, Tong, Anderson, arXiv:1412.0630 — exactly-sparse GP trajectory estimation.
- Hartikainen & Särkkä, Kalman filtering and smoothing solutions to temporal GP regression.
- Next: 06 — The SPDE approach (the PDE duality).

06 — The SPDE Approach (the PDE Duality)

What you'll learn: the exact sense in which a Matérn-covariance GP **is** the solution of a second-order stochastic PDE, why that operator is the place to inject physics, and how the finite-element/GMRF construction turns a dense covariance into a **sparse precision** matrix.

Prereqs: 01, 02;

05 gives the temporal analogue.

Why it matters here: this is the duality you asked about — the route to physical processes, boundary conditions, and cheap compute, all from one equation.

1. The duality in one equation

Whittle (1963) and Lindgren-Rue-Lindström (2011) showed: a Gaussian field with **Matérn covariance**

is the **stationary solution** of the linear **stochastic partial differential equation (SPDE)**

$$(\kappa^2 - \Delta)^{\alpha/2} X(s) = \eta(s)$$

- Δ is the **Laplacian** ($\sum \partial^2 / \partial s_d^2$) — the second-order differential operator. The name "duality

of the second-order PDE" points here.

- $\kappa > 0$ sets the **spatial range** ($\text{range} \approx \sqrt{(8\nu)/\kappa}$); larger $\kappa \rightarrow$ shorter correlations.
- $\alpha = \nu + d/2$ ties the SPDE order to the Matérn smoothness ν and the spatial dimension d .
- η is **Gaussian white noise** (the random forcing).

Read it as: "a Matérn GP is what you get when you pass white noise through the operator $(\kappa^2 - \Delta)^{-\alpha/2}$." The covariance function and the differential operator are two descriptions of the

same object — the duality. (This is the spatial sibling of doc 05:

there a Matérn GP in time equaled an ordinary SDE; here a Matérn GP in space equals a partial SDE.)

2. Why this is powerful — three doors from one equation

Because the GP is now defined by an **operator equation** rather than a fixed covariance formula, you

can edit the operator to change the model's assumptions:

- 1. Inject physical processes.** Replace/augment $(\kappa^2 - \Delta)$ with operators encoding real dynamics —

advection (transport by currents) and **anisotropic diffusion** (mixing). The resulting field automatically has the spatial/temporal correlation structure those physics imply. (Detailed in doc 07.)

- 2. Impose boundary conditions.** A PDE is only well-posed with boundary conditions. Specifying

Dirichlet/Neumann/Robin conditions on the domain boundary makes the GP respect walls, the

seabed, the surface, or known values — exactly, not approximately. (Also doc 07.)

3. Compute cheaply. Solving the SPDE by the **finite element method (FEM)** yields a Gaussian

Markov Random Field (GMRF) **with a** sparse precision matrix — the compute win, below.

3. From SPDE to GMRF: the sparse-precision win

The construction (LRL 2011):

1. Mesh the domain. Triangulate the region of interest (e.g. the operating area / water column)

into nodes and elements.

2. Basis expansion. Approximate the field as $X(s) \approx \sum_k w_k \phi_k(s)$ with local "tent" basis

functions ϕ_k (each nonzero only near node k) and random weights w .

3. Weak form / FEM assembly. Projecting the SPDE onto this basis gives the weights a Gaussian

distribution $w \sim N(0, Q^{-1})$ whose **precision matrix** Q is built from sparse FEM matrices (mass C and stiffness G), e.g. for $\alpha=2$: $Q = (\kappa^2 C + G) C^{-1} (\kappa^2 C + G)$.

Because each basis function overlaps only its neighbours, Q is **sparse** (mostly zeros). This is

the **Markov property in space**: a node is conditionally independent of the rest given its neighbours — the spatial analogue of the block-tridiagonal precision in doc 05 §3.

Dense covariance vs sparse precision — the cost table

	Covariance $\Sigma = K$ (classic GP)	Precision $Q = \Sigma^{-1}$ (GMRF/SPDE)
Structure	<code>**dense**</code> (every point covaries with every point)	<code>**sparse**</code> (only neighbours)
Storage	$O(n^2)$	$O(n)$
Factorize / solve	$O(n^3)$	$\approx O(n^{3/2})$ in 2-D, $O(n)$ in 1-D, using sparse Cholesky
Interpretation	"how outputs correlate"	"conditional dependence graph"

This is why the SPDE/GMRF approach scales to large spatial fields where a classic dense GP cannot — and why it is a serious **edge-compute** option (doc 08).

4. Non-stationarity for free

Let $\kappa = \kappa(s)$ and the diffusion vary over space, and the SPDE produces a **non-stationary** field —

different correlation length/strength in different regions — while keeping the sparse precision. Classic stationary kernels can't do this; encoding it in covariance form is hard, but in operator form

it's just spatially-varying coefficients. Underwater this matters: correlation structure near a seamount differs from open water.

5. Where it does and doesn't fit our project

- **Not the first build.** Our first GP is a temporal DVL-velocity model (doc 04); it doesn't need a spatial mesh.

- **The natural next step.** Modelling the **ocean-current field** the vehicle moves through is inherently spatial (and spatio-temporal). The SPDE/GMRF approach is the right tool: it can encode current physics (advection-diffusion), respect boundaries (seabed/surface), stay non-stationary, and compute on a sparse precision suitable for on-board use. This is item (2) of the edge/SPDE roadmap in implementation/04.
- **Spatio-temporal = combine docs 05 + 06.** Keep Markov-in-time (Kalman) and GMRF-in-space → an efficient current-field filter (Särkkä/Solin infinite-dimensional Kalman; Clarotto et al. for the advection-diffusion SPDE specifically).

6. Tooling note

The classic implementations live in the R-INLA ecosystem (R-INLA, inlabru) and fmesher; Python options include findiff/FEM libraries plus sparse solvers in scipy.sparse. For our roadmap this would be a separate component from the GPyTorch velocity model, justified only when we tackle the current field. Adding it implies a scipy-class dependency for sparse linear algebra and a meshing step.

Key takeaways

- **The duality:** a Matérn GP is the solution of $(\kappa^2 - \Delta)^{\alpha/2} X = \text{white noise}$. Covariance and second-order PDE operator are the same object described two ways.
- Editing the **operator** is how you inject **physics** and impose **boundary conditions** (see doc 07).
- Solving by FEM yields a **GMRF with a sparse precision matrix** → $O(n)$ -ish compute instead of $O(n^3)$, plus easy **non-stationarity**.
- For us: not the first build, but the principled, scalable way to model the **ocean-current field** later.

Further reading

- Lindgren, Rue, Lindström (2011), *An explicit link between Gaussian fields and Gaussian Markov random fields: the SPDE approach*, JRSS-B — **the paper**.
- Whittle (1963) — the original Matérn ↔ SPDE observation.
- Bakka et al. (2018), Spatial modelling with INLA: a review — accessible overview.
- Next: 07 — Physics-informed GPs and boundary conditions.

07 — Physics-Informed GPs and Boundary Conditions

What you'll learn: how to put real physics (advection-diffusion / ocean currents) and exact

boundary conditions (Dirichlet/Neumann/Robin) into a GP, via both the SPDE operator and

linear-PDE-constrained kernels.

Prereqs: 06 — the SPDE approach.

Why it matters here: this is the concrete "incorporate physical processes and boundary

conditions" ask; it underpins the future ocean-current-field model.

1. Two ways to make a GP obey physics

There are two complementary routes, both reducing to "constrain the GP with a **linear** differential operator":

- **A. The SPDE-operator route** (doc 06): change the operator on the left of $L X = \eta$ to encode the dynamics. The field's covariance then inherits the physics.
- **B. The constrained-kernel route:** start from a standard GP and **transform its kernel** so every sample exactly satisfies a linear PDE and/or boundary condition. If $f \sim \text{GP}(\theta, k)$ and $g = \eta f$ for a linear operator η , then g is also a GP with kernel $\eta \eta^T k$. So differential constraints map to kernel transformations — physics enters analytically.

Both work because **Gaussianity is preserved under linear operations** (differentiation, integration, linear PDE operators). Nonlinear physics needs approximation, but a lot of ocean transport is well captured by linear advection-diffusion.

2. The advection-diffusion SPDE (ocean currents)

The physical process most relevant to an AUV's drift is **advection-diffusion**: a quantity is **transported** by the current velocity field **and diffused** by mixing. The spatio-temporal SPDE (Clarotto, Allard, Romary, Desassis, 2022/2024) is:

$$\underbrace{\partial X / \partial t}_{\text{advection}} + \underbrace{(\gamma \cdot \nabla) X}_{\text{diffusion}} - \underbrace{\nabla \cdot (D \nabla X)}_{\text{damping}} + \underbrace{\kappa^2 X}_{\text{random forcing}} = \eta(s, t)$$

- γ — the **advection (drift) velocity** vector. Set it to the local current and the field's correlations **tilt and travel downstream** — features are carried along the flow, exactly like dye in a current. This is how the current itself enters the prior.

- D — the **diffusion tensor** (can be anisotropic): how fast structure smears out and in which directions (e.g. stronger horizontal than vertical mixing).

- κ^2 — damping/range term (as in doc 06).

- First-order $\partial/\partial t$ makes it a genuinely **nonseparable spatio-temporal** model — space and time

correlations are coupled, which separable kernels cannot represent.

Why this beats a plain space×time kernel

A naive spatio-temporal GP often uses a separable kernel $k_{\text{space}} \cdot k_{\text{time}}$, which assumes the

spatial pattern is frozen and just fades. Advection–diffusion produces **transported, evolving** correlations — physically correct for currents. That structure is the payoff of going physics-informed.

A numerical wrinkle: streamline diffusion

When advection dominates diffusion (fast current, little mixing), naive FEM discretization becomes

unstable/oscillatory. The standard fix is **streamline-diffusion (SUPG) stabilization** — add a small

amount of diffusion along the flow direction. Worth knowing because it's a required ingredient, not an

optional tweak, in advection-dominated regimes (Clarotto et al. discuss exactly this).

3. Boundary conditions — making the GP respect the domain

A PDE needs boundary conditions to be well-posed, and they are how the GP learns about walls, the

seabed, the surface, and known values. The three classical types:

Type	Constrains	Underwater example
Dirichlet	the value on the boundary: $X = g$	known current at an inflow boundary; zero anomaly at a reference
Neumann	the normal derivative : $\partial X / \partial n = h$	no flow through the seabed (zero normal velocity) — a hard physical fact
Robin	a mix: $aX + b \partial X / \partial n = c$	partially absorbing / leaky boundaries

Two ways to enforce them:

- **In the SPDE/FEM mesh:** impose the condition on boundary nodes during assembly — exact and natural

once you have a mesh. The choice of boundary condition genuinely changes the field near edges

(a Neumann "no-flow" seabed makes the modelled current go along the bottom, not through it).

- **In the kernel (boundary-constrained GPs):** design mean and covariance so **every** sample

satisfies the boundary condition by construction. Gulian et al. / Tan (JMLR v25, 23-1508) give closed-form **boundary-constrained kernels for Dirichlet, Neumann, Robin and mixed** conditions —

e.g. expanding the kernel in the **Dirichlet eigenfunctions of the Laplacian** enforces zero-value

boundaries globally and noiselessly. This route needs no mesh and is attractive on simple domains.

Why bother: enforcing physics-correct boundaries (a) removes nonsensical predictions near

edges,

(b) sharply reduces uncertainty there (the model "knows" the boundary), and (c) improves extrapolation into sparsely-observed regions — all valuable when bridging long DVL outages near the seabed.

4. Divergence-free / incompressibility constraints

Sea water is effectively **incompressible**: the current field should be **divergence-free** ($\nabla \cdot \mathbf{u} = 0$). Using the constrained-kernel route (§1B) you can build a **vector-field kernel** whose

samples are divergence-free by construction (curl-based / stream-function kernels).

Physics-informed,

boundary-constrained GP regression for fluid flow fields does exactly this (Jidling et al.; recent fluid-reconstruction work). For us this would make a reconstructed current field physically consistent,

which improves drift prediction.

5. How this would serve the AUV (the payoff)

Put together, a physics-informed current model gives the navigator a prior over the water it is moving through:

- During **DVL bottom-lock loss**, the vehicle still measures **water-relative** velocity (or nothing);

a current-field GP predicts the **current**, so ground velocity $\approx$ water-relative velocity + current

can be reconstructed $\rightarrow$ drift is bounded even without bottom-lock.

- Boundary conditions keep predictions sane near the seabed/surface.
- The SPDE/GMRF form (doc 06) keeps it computable, and combined

with Markov-in-time (doc 05) it can run as an efficient spatio-temporal filter.

6. Scope and honesty

This is the **most ambitious** thread and is **explicitly future work**

(implementation/04). It needs: a meshing/FEM or

constrained-kernel implementation, sparse linear algebra (scipy-class), an estimate of the current

field (γ) from data or a model, and validation data. The first build deliberately stays with the simpler, high-value temporal velocity GP.

Key takeaways

- Physics enters a GP through a **linear operator** — either by editing the **SPDE operator** (doc 06)

or by **transforming the kernel** ($g=f \Rightarrow k_g = f'k$); Gaussianity survives linear operations.

- **Advection-diffusion** encodes ocean currents: γ (drift) tilts/transport correlations, D (diffusion) smears them — a true nonseparable spatio-temporal model (needs streamline-diffusion)

stabilization when advection dominates).

- **Boundary conditions** (Dirichlet/Neumann/Robin) make the GP respect seabed/surface/walls exactly,

via the FEM mesh or **boundary-constrained kernels**; **divergence-free** kernels enforce incompressibility.

- Payoff: a physics-correct **current-field prior** that bounds drift during bottom-lock loss — future

work, but the principled end-state.

Further reading

- Clarotto et al. (2022/2024), *The SPDE approach for spatio-temporal datasets with advection and

diffusion*, arXiv:2208.14015.

- Gulian/Tan et al. (2024), Boundary-constrained Gaussian processes for ... linear PDEs, JMLR

v25/23-1508.

- Jidling et al., Linearly constrained Gaussian processes (divergence-free / curl-free kernels).
- Next: 08 — Cheap edge compute for GPs.

08 — Cheap Edge Compute for GPs

What you'll learn: the full menu of techniques for running GPs cheaply, their trade-offs, and a decision table for "what runs on the vehicle" vs "what we use ground-side."

Prereqs: 03 (the $O(n^3)$ problem),
05 (Kalman duality),
06 (GMRF/sparse precision).

Why it matters here: the eventual target is on-vehicle, GPU-free compute.

1. The problem restated

Exact GP inference is $O(n^3)$ time, $O(n^2)$ memory (doc 03).

On a vehicle CPU with no GPU, hard real-time, and a power budget, that's a non-starter for large n .

The good news: for the structured problems we have (temporal, or spatial-with-locality), there are

exact linear-time methods — we don't even need approximations for the first build.

2. The menu (cheapest path first for our case)

(a) Sliding window — trivial, used for the demo

Keep only the last W seconds of DVL data ($n \approx 60\text{--}180$ at ~ 1 Hz). $O(n^3)$ on $n \approx 100$ is microseconds.

Refit hyperparameters every `refit_every` samples, not every sample. This alone makes the demo

real-time-trivial. Limitation: forgets old data (fine — velocity dynamics are local).

(b) State-space / Kalman GP — exact, $O(n)$, the edge winner for temporal

A Matérn temporal GP **is** a Kalman filter (doc 05):

- $O(n)$ time, $O(1)$ memory per step (recursive), no matrix to store.
- Exact (not approximate) for Matérn kernels.
- Naturally streaming and asynchronous; closed-form interpolation.
- Maps onto imu-drift's `no_std` Rust → literally embeddable.

This is the recommended edge form for the velocity model.

(c) GMRF / SPDE sparse precision — exact-ish, $O(n)$ - $O(n^{\{3/2\}})$ for spatial

For spatial/spatio-temporal fields (the current model), the SPDE→GMRF construction gives a sparse

precision matrix solved with sparse Cholesky (doc 06).

Scales where dense GPs can't; the spatial counterpart of (b).

(d) Inducing-point / sparse approximations — for big, unstructured data

When data is large and lacks Markov structure, summarize it with $m \ll n$ **inducing points**:

- **FITC/SoR/DTC** (Snelson & Ghahramani; Quiñonero-Candela & Rasmussen): $O(n m^2)$.
- **SVGP / variational sparse GPs** (Titsias; Hensman): mini-batch trainable, $O(m^3)$ per step, the

modern default in GPyTorch for scale.

Approximate (quality depends on m and inducing-point placement). Probably unnecessary for us given

(b), but the standard tool if structure is absent.

(e) Structured-kernel tricks — for gridded data

KISS-GP / SKI (Wilson & Nickisch) exploit Kronecker/Toeplitz structure on grids for near-linear cost. Niche for us (our data isn't gridded) but powerful where it applies.

(f) Implementation-level levers (orthogonal to the above)

- **Single precision (float32)** — $\sim 2\times$ speed/memory; usually fine for navigation.
- **Fixed-lag smoothing** — smooth only the last L steps for bounded latency/cost.
- **Quantization / fixed-point** — for microcontroller targets, on the Kalman recursion.
- **Precompute & freeze hyperparameters** — learn θ offline from logs, run inference-only on-board

(hyperparameters drift slowly; no need to optimize on the vehicle).

- **Reuse the Cholesky factor** across mean/variance/marginal-likelihood (already standard).

3. Decision table — our project

Component	Where	Method	Cost	Deps
DVL-velocity GP — demo	ground-side / dev	sliding-window exact GP (GPyTorch)	$O(n^3)$, $n \approx 100 \rightarrow$ trivial	torch/gpytorch
DVL-velocity GP — edge	on vehicle	state-space / Kalman GP (method b)	$O(n)$, $O(1)$ mem	numpy or 'no_std' Rust
Ocean-current field (future)	on vehicle / ground	SPDE-GMRF (method c), Markov-in-time	$O(n) - O(n^{3/2})$	scipy.sparse / FEM
Hyperparameter learning	offline	marginal-likelihood + Adam, then freeze	one-off	torch/gpytorch

Punchline: we get the best of both worlds with **no accuracy compromise** — develop and tune with

the flexible GPyTorch batch GP now, then deploy the mathematically equivalent $O(n)$ Kalman form on the

vehicle, with hyperparameters frozen from logs (or taken from imu-drift's Allan-variance fit).

4. What we deliberately avoid

- No GPU dependency on-vehicle (the edge form is CPU/microcontroller-friendly).
- No giant unstructured dense GP (we always have temporal or spatial structure to exploit).
- No online hyperparameter optimization on-board (freeze them; they move slowly).

5. Sizing intuition

- State-space Matérn-3/2 velocity GP: state ≈ 2 per axis $\times 3$ axes $\approx 6-9$ floats; per-step cost is a

handful of tiny matrix ops — **negligible** next to the existing 16-state EKF.

- Sliding-window batch GP at $n=120$, 3 outputs (multitask 360×360): a few ms per refit on a laptop —

fine for the demo, refit every few seconds.

- A 2-D current GMRF with $\sim 10^3-10^4$ mesh nodes: seconds per solve with sparse Cholesky

ground-side;
needs the state-space-in-time trick for on-board real-time.

Key takeaways

- For **temporal** GPs (our velocity model), the **state-space/Kalman** form is **exact and $O(n)$** —

the edge solution, no approximation needed.

- For **spatial** fields (the current model), **SPDE/GMRF sparse precision** is the analogue.
- **Inducing points (SVGP/FITC)** are the fallback only when there's no structure — we have structure,

so we likely won't need them.

- Strategy: tune flexibly off-board (GPyTorch), deploy the equivalent $O(n)$ form on-board with frozen

hyperparameters. Same accuracy, tiny cost.

Further reading

- Quiñero-Candela & Rasmussen (2005), unifying view of sparse approximate GPs.
- Hensman, Fusi, Lawrence (2013), Gaussian Processes for Big Data (SVGP).
- Wilson & Nickisch (2015), KISS-GP.
- Särkkä, Solin, Hartikainen (2013), *Spatiotemporal learning via infinite-dimensional Bayesian

filtering and smoothing* (state-space GPs).

- Next: 09 — Glossary and annotated references.

09 — Glossary and Annotated References

What you'll learn: quick definitions of the recurring terms, and an annotated, linked bibliography of every source with a one-line "read this for X."

Prereqs: none (reference doc).

Why it matters here: a single place to look things up and to find the primary sources.

Glossary

- **Gaussian Process (GP)** — a distribution over functions; any finite set of outputs is jointly

Gaussian, defined by a mean function and a kernel. doc 01

- **Kernel / covariance function** $k(\mathbf{x}, \mathbf{x}')$ — encodes how correlated two outputs are; sets

smoothness, amplitude, reach. doc 02

- **Matérn kernel** — kernel family with tunable smoothness ν ; $\nu=3/2$ (once-differentiable) is our

default. Exactly representable as an SDE/Kalman model and the SPDE Laplacian model. 02, 05, 06

- **Lengthscale** ℓ — input distance over which outputs stay correlated; the main smoothness knob.
- **Signal variance** σ^2 / **noise variance** σ_n^2 — amplitude of the function / of observation noise.
- **ARD** — automatic relevance determination; per-input lengthscales that let the model ignore

irrelevant inputs. 02

- **Multitask / multi-output GP (LMC/ICM)** — joint GP over correlated outputs via a coregionalization

matrix B ; used for correlated (v_x, v_y, v_z) . 02 §8

- **Marginal likelihood** — probability of the data under the GP prior; maximized to learn hyperparameters; auto-balances fit vs complexity. 01 §5

- **Predictive mean / covariance** — the GP's outputs at a query point; the covariance is what we feed

the EKF as adaptive R. 03 §6

- **Cholesky factorization** — stable LL^T decomposition used to solve GP equations; the $O(n^3)$ step.

03

- **Jitter** — tiny diagonal added for numerical stability. 03 §5
- **Error-state (indirect) EKF** — filters the small error between truth and an IMU-propagated nominal

state; this repo's filter. 04 §1

- **Adaptive measurement noise** R_k — using a per-measurement, GP-predicted covariance instead of a

fixed R ; the core fusion idea. 04 §2-3

- **Mahalanobis distance** — covariance-aware distance for outlier gating. 04 §2
- **DVL (Doppler Velocity Log)** — acoustic sensor giving body-frame velocity (here, resolved 3-D);

loses **bottom-lock** over soft/steep terrain → dropouts.

- **Dead-reckoning** — integrating velocity/acceleration to get position; drifts without absolute fixes.

- **SDE (stochastic differential equation)** — differential equation driven by white noise; a temporal

GP's state-space form. 05

- **State-space GP / Kalman duality** — Markovian temporal GP $\equiv$ Kalman filter/RTS smoother; $O(n)$.

05

- **Precision matrix** $\mathbf{Q} = \mathbf{\Sigma}^{-1}$ — inverse covariance; sparse for Markov processes (block-tridiagonal

in time, neighbour-sparse in space). 05 §3, 06 §3

- **SPDE (stochastic PDE)** — PDE with random forcing; $(\kappa^2 - \Delta)^{\alpha/2} X = ?$ has a Matérn solution — the

PDE $\leftrightarrow$ GP duality. 06

- **Laplacian** Δ — the second-order operator $\Sigma \partial^2 / \partial s^2$; the "second-order PDE" in the duality.
- **GMRF (Gaussian Markov Random Field)** — Gaussian field with sparse precision; the FEM solution of

the SPDE. 06

- **FEM (finite element method)** — mesh-and-basis discretization that turns the SPDE into a GMRF.
- **Advection / diffusion** — transport by a velocity field / spreading by mixing; the physics encoded

in the current-field SPDE. 07 §2

- **Dirichlet / Neumann / Robin boundary conditions** — fix value / normal-derivative / a mix on the

domain boundary. 07 §3

- **Inducing points (FITC/SVGP)** — $m \ll n$ summary points for approximate large-scale GPs.

08 §2d

- **GPyTorch** — PyTorch-based GP library (autodiff hyperparameters, multitask kernels, SVGP); our

demo-build tool.

Annotated references

GPs for underwater / AUV navigation

- Cohen & Klein (2025), Gaussian Process Regression for Improved Underwater Navigation —

arXiv:2502.16510. Read for: the exact pattern we follow — MOGPR on DVL feeding adaptive R_k into

an error-state EKF; $\sim 20\text{--}42\%$ gains; composite ARD-SE+Matérn $3/2$ +RQ kernel; real-time cost caveat.

<<https://arxiv.org/abs/2502.16510>>

- **Position Estimation for UVs using UKF with GP Prediction (GP-UKF)** — Read for: learning the

motion/observation model with a GP inside a UKF; graceful degradation.

- Ko & Fox, GP-BayesFilters — Read for: the foundational GP-in-Bayes-filter framework.

Continuous-time trajectory estimation / state-space GPs

- Barfoot, Tong, Anderson (2014), *Batch Continuous-Time Trajectory Estimation as Exactly Sparse GP

Regression* — arXiv:1412.0630. Read for: the block-tridiagonal precision, white-noise-on-

acceleration prior, $O(n)$ trajectory estimation, query/interpolation.

<<https://arxiv.org/abs/1412.0630>>

- Särkkä, Solin, Hartikainen (2013), *Spatiotemporal learning via infinite-dimensional Bayesian

filtering and smoothing* — Read for: the kernel $\leftrightarrow$ SDE $\leftrightarrow$ Kalman equivalence and $O(n)$ temporal GPs.

- Särkkä & Solin, Applied Stochastic Differential Equations (2019, free) — Read for: the math

of turning kernels into SDEs. <<https://users.aalto.fi/~ssarkka/>>

- Hartikainen & Särkkä (2010), Kalman filtering and smoothing solutions to temporal GP regression.*

The SPDE / PDE duality

- Lindgren, Rue, Lindström (2011), *An explicit link between Gaussian fields and Gaussian Markov

random fields: the SPDE approach*, JRSS-B 73(4) — Read for: the core duality and the FEM $\rightarrow$ GMRF

sparse-precision construction.

<<https://portal.research.lu.se/en/publications/an-explicit-link-between-gaussian-fields-and-gaussian-markov-rand/>>

- **Whittle (1963)** — Read for: the original Matérn $\leftrightarrow$ SPDE observation.
- Bakka et al. (2018), Spatial modelling with INLA: a review — Read for: an accessible tour of

SPDE/GMRF modelling and the R-INLA tooling.

• Lindgren, Bolin, Rue (2022), The SPDE approach ... 10 years and still running, Spatial Statistics — Read for: the modern state of the field.

Physics-informed GPs and boundary conditions

- Clarotto, Allard, Romary, Desassis (2022/2024), *The SPDE approach for spatio-temporal datasets

with advection and diffusion*, Spatial Statistics — arXiv:2208.14015. Read for: the advection-diffusion SPDE (currents), nonseparability, streamline-diffusion stabilization.

<<https://arxiv.org/abs/2208.14015>>

- Gulian / Tan et al. (2024), *Boundary-constrained Gaussian processes for robust physics-informed

ML of linear PDEs*, JMLR 25 — Read for: exact Dirichlet/Neumann/Robin-constrained mean+kernel

design. <<https://www.jmlr.org/papers/v25/23-1508.html>>

• Jidling, Wahlström, Wills, Schön, Linearly constrained Gaussian processes — Read for: divergence-free / curl-free vector-field kernels (incompressible currents).

- **Physics-informed, boundary-constrained GPR for fluid flow fields (2025), arXiv:2507.17582** —

Read for: a worked fluid-reconstruction example with incompressibility + boundaries.

Scalable / sparse GPs (edge compute)

- Rasmussen & Williams (2006), Gaussian Processes for Machine Learning (free) — Read for:

everything foundational (chs. 2-5). <<https://gaussianprocess.org/gpml/>>

- **Quiñonero-Candela & Rasmussen (2005)** — Read for: a unifying view of sparse approximate GPs

(SoR/FITC/DTC).

- Hensman, Fusi, Lawrence (2013), Gaussian Processes for Big Data — Read for: SVGP / variational

sparse GPs.

- Wilson & Nickisch (2015), KISS-GP — Read for: structured-kernel near-linear inference on grids.
- **GPyTorch (Gardner et al., 2018) + docs** — Read for: the library we'll use; multitask GPs, SVGP,

exact GP tutorials. <<https://gpytorch.ai/>>

Further reading

- Duvenaud, The Kernel Cookbook — kernel-composition intuition.
<<https://www.cs.toronto.edu/~duvenaud/cookbook/>>
- A Visual Exploration of Gaussian Processes (distill.pub) — interactive intuition.
- Back to the overview or on to the

implementation plan.

00 — Implementation Plan: DVL-Dropout Velocity GP

What you'll learn: the concrete first build, end to end.

Prereqs: references/04 (why);

this doc is the what/how.

Why it matters here: this is the spec for the coding round (separately approved).

Goal

Add a Gaussian Process that models the **DVL body-frame velocity over time** and uses it to **denoise, adaptively weight** (R_k), **reject outliers from**, and **bridge dropouts in** the DVL stream feeding the existing error-state EKF — with **zero changes to the EKF core**.

Confirmed posture: **demo-first, accuracy over optimization now; eventual edge**; use a GP library

(GPyTorch). The module is built behind a `predict()/ingest()` interface so the GPyTorch engine can

later be swapped for the $O(n)$ state-space form (references/05, 04-edge-and-spde-roadmap).

Why it needs no EKF-core changes (the key seam)

The existing EKF already accepts a **per-measurement covariance** and already does outlier gating:

- `CorrectedDvlMeasurement` carries `velocity_body_m_s` **and** `covariance_body` (`dvl_correction/src/dvl_imu_kalman.py:38-50`).
- `update_corrected_dvl_velocity` builds R from that covariance via `_velocity_measurement_covariance(...)` (`dvl_imu_kalman.py:213-230`).
- `_apply_measurement_update` already rejects gross outliers with a **Mahalanobis gate** (`dvl_imu_kalman.py:277-300`).

So the GP injects results purely by constructing a

``CorrectedDvlMeasurement(velocity_body_m_s=..., covariance_body=...)`. Adaptive  $R_k$  = "set `covariance_body`` to the GP predictive covariance." See

01-codebase-architecture-and-seams for exact anchors.

Components to build

1. ``dvl_correction/src/gp_velocity.py`` — `GpVelocityConfig` + `GpDvlVelocityModel` with

`ingest / fit / predict / is_outlier`. Multitask (Matérn-3/2 + RQ + RBF) kernel over time → 3-D

velocity, sliding window, periodic refit, dropout-bridge logic. Full design in 02-gp-velocity-module-design.

2. **Pipeline wiring** in `dvl_correction/src/synchronization.py` — an optional `gp_velocity`

component (parallel to the existing `dvl_track`), doing denoise + adaptive- R on accepted DVL and

injecting bridge measurements during silence. Anchors in 01-codebase-architecture-and-seams.

3. Config + exports — a GP block in `dvl_correction/src/config.py` build_navigation_pipeline

(off by default), and exports from `dvl_correction/src/__init__.py`.

4. Dependency — optional extra in `dvl_correction/pyproject.toml`:
[project.optional-dependencies] gp = ["gpytorch>=1.11", "torch>=2.1"];
`gp_velocity.py` imports
`torch/gpytorch lazily` so the core stays numpy-only.

5. Benchmark + demo — `examples/gp_dropout_benchmark.py` (with/without-GP RMSE) and optional

`examples/gp_demo_ws.py` (live frontend). See 03-benchmark-and-frontend-demo.

6. Tests — `dvl_correction/tests/test_gp_velocity.py` (unit) + a pipeline integration test.

The three usage modes (recap)

Mode	Trigger	Action	Effect
Denoise + adaptive 'Rk'	a valid DVL sample arrives	replace velocity with GP mean; set 'covariance_body' = GP cov	cleaner, correctly-weighted updates
Outlier rejection	incoming sample far from GP band	drop before fusion (EKF gate is backstop)	robustness
Dropout bridge	DVL silent > 'bridge_after_s'	inject 'CorrectedDvlMeasurement' from GP prediction, cov grows with gap	graceful degradation to IMU

Build order (suggested)

1. `gp_velocity.py` standalone + unit tests (no pipeline yet) — verify mean/cov, variance growth, outlier flag.
2. Synthetic benchmark harness — generate truth + DVL with dropouts/outliers.
3. Pipeline wiring (denoise + adaptive R first; bridge second), off by default.
4. Run benchmark with/without GP → record RMSE + plot.
5. (Stretch) WebSocket bridge to the React viewer.
6. (Later round) state-space edge port; (future) SPDE current field.

Dependencies & risk

- **New deps:** gpytorch, torch as an **opt-in extra** only. Core unaffected; existing numpy-only tests keep passing (GP off by default).
- **Risk:** torch is heavy and **not** edge-deployable — accepted for the demo; the edge port (04-edge-and-spde-roadmap) removes it. Lighter fallback: `sklearn.gaussian_process` per-axis if torch is unwanted.
- **Risk:** over-smoothing / over-confident bridging — mitigated by Matérn-3/2, learned lengthscale, `max_bridge_s`, and benchmark validation (references/04 §7).

Verification (summary; full detail in doc 03)

1. **Unit:** finite 3-vector mean + SPD 3×3 cov; predictive variance grows with extrapolation; `is_outlier` flags an injected spike; passthrough below `min_samples`.
2. **Integration:** GP-enabled pipeline output shapes identical to baseline; never raises;

`gp_bridge_updates > 0` during a simulated dropout.

3. Benchmark: GP-enabled position RMSE measurably below baseline across dropout windows.

4. Regression: `python3 -m unittest discover -s dvl_correction/tests` stays green (GP off by default).

5. Stretch: `WS bridge + npm run dev` shows a tight covariance ellipse through a dropout.

Key takeaways

- One module + thin pipeline wiring; **EKF untouched** thanks to the `covariance_body` seam.
- GPyTorch now for accuracy; identical interface enables the $O(n)$ edge swap later.
- Validated by a synthetic dropout benchmark and (optionally) the existing covariance-ellipse frontend.

Further reading

- 01 — Codebase architecture and seams
- 02 — `gp_velocity` module design

01 — Codebase Architecture and Seams

What you'll learn: the relevant files, the exact integration points (with file:line), the data

flow, and the frame conventions you must respect.

Prereqs: 00-implementation-plan.

Why it matters here: so the GP code slots in correctly and safely.

1. The packages

- ``dvl_correction`` — Python (numpy-only), the navigation fusion library. All GP work lives here. `pip install -e ./dvl_correction; tests python3 -m unittest discover -s dvl_correction/tests`.

- ``imu-drift`` — Rust `no_std` crate, closed-form IMU drift mean/variance + Allan variance. Not wired into Python; relevant as the **edge GP prior** (references/05 §5).

- ``frontend`` — React/TS viewer + Rust WS stub server. Relevant only for the optional live demo (03-benchmark-and-frontend-demo).

2. Data flow (current)

```
raw sensors → NavigationFusionPipeline (synchronization.py)
               | buffers events in a timestamp-ordered heap (max_delay_s watermark)
               | interpolates IMU between samples
               ▼
DvlCorrectionLayer.correct (dvl_correction.py) — frame/scale/lever-arm + quality gates
               | → CorrectedDvlMeasurement(velocity_body_m_s, covariance_body, ...)
               ▼
(optional) DvlDeadReckoningTrack (dead_reckoning.py) — velocity → nav-position pseudo-measurement
               ▼
DvlImuKalmanLayer (dvl_imu_kalman.py) — error-state EKF: propagate (IMU) + update (DVL/mag)
               ▼
NavigationOutput (per step: position, velocity, attitude, biases, 16×16 covariance)
```

The GP inserts between "corrected DVL" and "EKF update," as a sibling to `DvlDeadReckoningTrack`.

3. The exact seams

Seam A — the measurement carries its own covariance (enables adaptive R_k)

`dvl_correction/src/dvl_imu_kalman.py:38-50:`

```
@dataclass(frozen=True)
class CorrectedDvlMeasurement:
    timestamp_s: float
    velocity_body_m_s: ArrayLike3 | None = None
    covariance_body: CovarianceLike3 | None = None      # ← set this to the GP predictive cov
    position_nav_m: ArrayLike3 | None = None
    position_covariance_nav: CovarianceLike3 | None = None
```


The velocity update consumes it (dvl_imu_kalman.py:213-230):

```
def update_corrected_dvl_velocity(self, measurement):
    velocity_body = _as_vec3(measurement.velocity_body_m_s, "velocity_body_m_s")
    measurement_covariance = _velocity_measurement_covariance(measurement, self.config) # ← uses covariance_body
    ...
    return self._apply_measurement_update(residual, h_matrix, measurement_covariance)
```

So **adaptive R_k** requires no EKF change — just populate covariance_body.

Seam B — outlier gating already exists

dvl_imu_kalman.py:277-300 (_apply_measurement_update) computes the innovation covariance and rejects updates whose **Mahalanobis distance** exceeds config.mahalanobis_gate (default 16.27). The GP's is_outlier is an additional, earlier pre-filter; this is the backstop.

Seam C — the pipeline injection points

dvl_correction/src/synchronization.py:

- _process_event handles raw_dvl (lines 136-149) and corrected_dvl (127-135). After correction

and before self.kalman.update_corrected_dvl(...), call the GP to **denoise + set covariance**.

- _add_dvl_track_position (lines 158-177) is the **template**: an optional component (self.dvl_track) that transforms/augments a CorrectedDvlMeasurement before it reaches the EKF.

The GP component mirrors this exactly.

- _propagate_filter_to (lines 179-185) advances the filter to a timestamp via interpolated IMU;

this is where, during DVL silence, the **bridge** logic injects a GP-predicted velocity update.

- Add a gp_velocity constructor arg next to dvl_track (lines 46-56), and counters to PipelineDiagnostics (lines 24-35): gp_denoised, gp_bridge_updates, gp_outlier_rejections.

Seam D — the dead-reckoning template (shape to mirror)

dvl_correction/src/dead_reckoning.py — DvlTrackConfig / DvlTrackState / DvlDeadReckoningTrack

with update(...) returning a state carrying position_nav_m + covariance_nav. The GP module copies

this dataclass-config + state + helper structure (and the numpy _as_vec3 / _as_covariance / _quat_to_rotation_matrix helpers, lines 91-122).

Seam E — config + exports

- dvl_correction/src/config.py build_navigation_pipeline has a _dvl_track block; add an analogous

gp_velocity block, **off by default**.

- dvl_correction/src/__init__.py exports the public symbols; add GpDvlVelocityModel,

GpVelocityConfig.

4. Frame & convention rules (do not violate)

- **DVL velocity is body-frame** (`velocity_body_m_s`); **position is nav-frame** (ENU). The GP models

velocity in **body frame** (same as the measurement) so its mean/cov drop straight into `update_corrected_dvl_velocity` with no rotation. (If you ever rotate a covariance, use $R \Sigma R^T$ as `dead_reckoning.py:70-74` does.)

- **Attitude is a wxyz quaternion** (`NavigationState.attitude_quat_wxyz`).

• **Timestamps are monotonic**; the pipeline rejects out-of-order events (`synchronization.py:103-108`). The GP must query/ingest in time order — it already receives events in order.

- **Covariances must be SPD**; symmetrize on output (the repo does $0.5*(C+C^T)$).

5. Tests to mirror

- `dvl_correction/tests/test_ekf_quality.py` — `_run_stationary_with_accel_bias` builds a synthetic

IMU+DVL scenario and asserts on drift; the GP benchmark reuses this pattern with dropouts.

- `dvl_correction/tests/test_dead_reckoning_and_cli.py` — pattern for testing a pipeline component.

Key takeaways

- Insert the GP **between corrected DVL and the EKF update**, mirroring `DvlDeadReckoningTrack`.
- **Adaptive** `Rk` = `set CorrectedDvlMeasurement.covariance_body`; **outlier gate** already exists.
- Wiring points: `synchronization.py _process_event / _add_dvl_track_position / _propagate_filter_to`; config in `config.py`; exports in `__init__.py`.
- Respect body/nav frames, wxyz quaternion, monotonic time, SPD covariances.

Further reading

- 02 — `gp_velocity` module design

02 — gp_velocity Module Design

What you'll learn: the full design of `dvl_correction/src/gp_velocity.py` — config, API, kernel, windowing, refit, and dropout-bridge logic.

Prereqs: [references/02-03](#),
[01-codebase-architecture-and-seams](#).

Why it matters here: this is the buildable spec for the new module.

1. Public API (engine-agnostic)

The module is designed so the **engine** (GPyTorch now, state-space Kalman later) hides behind one interface. Pipeline code only ever calls these:

```
@dataclass(frozen=True)
class GpVelocityConfig:
    window_seconds: float = 120.0      # sliding-window length of DVL history
    min_samples: int = 12              # below this, passthrough (no GP)
    refit_every: int = 10              # refit hyperparameters every N ingests
    bridge_after_s: float = 1.5        # DVL silence before we start bridging ( $\approx >1$  DVL period)
    bridge_cadence_s: float = 1.0     # how often to inject a bridge update during silence
    max_bridge_s: float = 20.0         # stop bridging after this (let IMU take over fully)
    outlier_mahalanobis: float = 9.0   # GP pre-filter threshold ( $3\sigma$  in 3-D  $\approx 9$ )
    jitter: float = 1e-6
    # kernel/init hyperparameters (seeds; learned by fit)
    init_lengthscales: float = 5.0
    init_noise_std_m_s: float = 0.20  # ~ KalmanConfig.default_dvl_velocity_std_m_s

class GpDvlVelocityModel:
    def __init__(self, config: GpVelocityConfig | None = None) -> None: ...

    def ingest(self, timestamp_s: float,
               velocity_body_m_s, covariance_body=None) -> None:
        """Append an accepted DVL velocity to the sliding window."""

    def fit(self) -> None:
        """(Re)optimize hyperparameters on the current window (called internally per refit_every)."""

    def predict(self, timestamp_s: float) -> tuple[np.ndarray, np.ndarray]:
        """Return (mean3, cov3x3) body-frame velocity at a query time.
        Variance grows with extrapolation distance from the window."""

    def is_outlier(self, timestamp_s: float, velocity_body_m_s) -> bool:
        """Mahalanobis distance of a candidate vs the GP predictive band > threshold."""

    @property
    def ready(self) -> bool:
        """True once  $\geq$  min_samples are in the window (else callers should passthrough)."""

    @property
    def last_ingest_s(self) -> float | None: ...
```

Return types are plain numpy (mean3 shape (3,), cov3x3 shape (3,3), SPD) so the pipeline

can

build a `CorrectedDvlMeasurement` directly — no torch types leak out.

2. Internal state

- A ring buffer of `(t, v_body[3], per_sample_var[3] | None)` trimmed to `window_seconds`.
- Cached fitted model/hyperparameters; an ingest counter for `refit_every`.
- **Lazy imports:** `import torch, gpytorch` happen inside methods, so importing `dvl_correction` without the `[gp]` extra never fails. If unavailable and the model is used, raise a clear error pointing at `pip install -e ./dvl_correction[gp]`.

3. The model (GPyTorch, multitask)

- **Inputs:** time, **centered** on the window start and scaled (numerical conditioning, references/03 §7).
- **Outputs:** 3-D body-frame velocity, **standardized** per axis (undo on output).
- **Kernel:** ``MultitaskKernel(base = ScaleKernel(MaternKernel(v=1.5)) + ScaleKernel(RQKernel())``
 - `ScaleKernel(RBFKernel()), num_tasks=3, rank=...)` — the composite from references/02 §7-8: Matérn-3/2 (physical smoothness) + RQ (multi-scale) + RBF (trend), with a learned 3×3 task covariance for cross-axis correlation.
- **Likelihood:** `MultitaskGaussianLikelihood`; when per-sample covariance_{body} is supplied, use a **fixed/added per-point noise** so noisier DVL samples are trusted less (references/03 §4).
- **Training (`fit`):** maximize the exact marginal likelihood with Adam for a small fixed number of iterations on the window; warm-start from the previous fit's hyperparameters for speed/stability.

Lighter fallback (if torch is unwanted): three independent `sklearn.gaussian_process.GaussianProcessRegressors` (one per axis) with `Matern(v=1.5)+RationalQuadratic+RBF`, `return_cov` for the diagonal. Loses cross-axis correlation but keeps the same external API.

4. predict() semantics (the important behaviours)

- **Inside the window / near data:** posterior mean denoises; covariance is small.
- **Short extrapolation (a dropout just started):** mean continues the trend; covariance modestly inflated.
- **Long extrapolation:** covariance climbs toward the prior, so downstream weighting collapses gracefully. This is automatic from the GP math — we don't hand-code variance inflation.
- Always returns an **SPD** cov3x3 (add `jitter*I`, symmetrize).

5. Dropout-bridge logic (where modes combine)

Pseudo-logic the pipeline drives (see 01 §3 Seam C):

```
on accepted DVL sample at t, v:
    if model.ready and not model.is_outlier(t, v):
        m, C = model.predict(t)          # denoise
        emit CorrectedDvlMeasurement(t, velocity_body_m_s=m, covariance_body=C)    # adaptive R
        model.ingest(t, v, meas_cov)
    elif model.ready and model.is_outlier(t, v):
        diagnostics.gp_outlier_rejections += 1    # drop; EKF gate is backstop
    else:
        emit the raw corrected measurement unchanged    # passthrough until min_samples
        model.ingest(t, v, meas_cov)

during DVL silence (driven in _propagate_filter_to):
    gap = now - model.last_ingest_s
    if model.ready and bridge_after_s < gap <= max_bridge_s, every bridge_cadence_s:
        m, C = model.predict(now)          # C grows with gap
        emit CorrectedDvlMeasurement(now, velocity_body_m_s=m, covariance_body=C)
        diagnostics.gp_bridge_updates += 1
    # gap > max_bridge_s: stop bridging, let IMU dead-reckoning run
```

Note: bridge measurements are **not** re-ingested (they are model outputs, not observations) — avoids the GP feeding on its own predictions.

6. Refit cadence & cost

- Refit on the window every `refit_every` ingests (not every sample): hyperparameters move slowly (references/03 §8).
- With `window_seconds=120` at ~ 1 Hz, $n \approx 120$, multitask 360×360 — a few ms per refit on a laptop.

Predict is sub-ms. Real-time-trivial for the demo (references/08 §5).

7. Designed for the edge swap

`predict()/ingest()` is exactly the interface a **state-space Matérn GP** implements (references/05). The edge version replaces the GPyTorch internals with a tiny Kalman recursion (state $\approx 2/\text{axis}$), keeps the same external behaviour, drops torch, and can take its process noise from imu-drift's Allan-variance fit. No pipeline changes when swapping. Roadmap: 04-edge-and-spde-roadmap.

8. Edge cases to handle

- Fewer than `min_samples` → `ready=False`, passthrough.
- Duplicate/near-equal timestamps → drop the duplicate before fit (jitter won't save a singular K).
- All-window-outliers → fit on what's left; rely on EKF gate; consider a robust loss later.
- NaN/inf guard on inputs and on predict outputs (return passthrough + log if the fit is degenerate).

Key takeaways

- One class, engine-agnostic ingest/fit/predict/is_outlier, numpy in/out, lazy torch import.
- Multitask **Matérn-3/2 + RQ + RBF** kernel over centered time → standardized 3-D velocity, with per-sample noise.
 - Denoise + adaptive R + outlier pre-filter + **gap bridging with auto-growing variance**, capped by `max_bridge_s`; bridge outputs are not re-ingested.
 - Same interface enables the **$O(n)$ edge swap** with zero pipeline change.

Further reading

- 03 — Benchmark and frontend demo
- GPyTorch multitask GP regression tutorial (gpytorch.ai).

03 — Benchmark and Frontend Demo

What you'll learn: how to prove the GP helps (synthetic dropout benchmark with RMSE) and how to show it live in the existing React covariance-ellipse viewer.

Prereqs: 00–02.

Why it matters here: "demo-first, accuracy" means the benchmark is the primary deliverable.

1. Why synthetic data

The repo has **no ground-truth datasets** — only synthetic scenarios inside tests (dvl_correction/tests/test_ekf_quality.py). Ground truth is what lets us measure accuracy, so we generate it: a known trajectory → simulate sensors from it → estimate → compare to truth.

2. The benchmark: examples/gp_dropout_benchmark.py

Generate ground truth

A smooth 3-D trajectory (e.g. a gentle lawnmower or sinusoidal path) sampled at high rate; from it compute true position, velocity (body & nav), and attitude.

Simulate sensors (reuse the test pattern)

- **IMU @ ~100 Hz:** specific force + angular rate implied by the trajectory + bias + noise (mirror `_run_stationary_with_accel_bias` in tests/test_ekf_quality.py).
- **DVL @ ~1 Hz:** body-frame velocity + noise, then **corrupt**:
 - inject **outlier spikes** at a few timestamps (e.g. $\times 5$ magnitude), and
 - one or more **dropout windows** (e.g. 10 s and 30 s) where no DVL is emitted.
- (Optional) magnetometer yaw, as in tests/test_magnetometer.py.

Run both configurations

Feed identical sensor streams to two NavigationFusionPipelines:

- **baseline:** `gp_velocity=None`.
- **GP:** `gp_velocity=GpDvlVelocityModel(...)` enabled.

Metrics & artifacts

- **Position RMSE** and **max error** vs truth, overall and **restricted to the dropout windows**

(where the GP should help most).

- Velocity RMSE; orientation error (the literature shows orientation also improves).
- Pipeline diagnostics: `gp_denoised`, `gp_bridge_updates`, `gp_outlier_rejections`, `dvl_gate_rejections`.

- A **matplotlib** figure: true path vs baseline vs GP (top-down), plus a covariance-vs-time panel

showing the GP's covariance growing through the dropout while position error stays bounded.

- Print a small table and save the plot to `examples/out/`.

Success criterion

GP-enabled **position RMSE during/after dropouts is measurably lower** than baseline, with no regression on clean segments. Record the numbers in the PR.

3. Optional live demo: the existing frontend

The React app already renders a position **mean + covariance ellipse** and speaks a fixed WebSocket contract — so a real GP posterior drops in with **no frontend code changes**.

The contract (already defined)

- frontend/web/src/types.ts and frontend/server/src/main.rs: server→client at ~20 Hz emits
tick { t, mean:[x,y], cov:[[xx,xy],[xy,yy]] } and status { state }; client→server sends
set_path, play, pause, reset.
- The covariance ellipse is rendered by frontend/web/src/canvas/ellipse.ts (ellipseFromCov).
- The current server/ is an explicit **stub** (struct Sim) that grows covariance linearly — the
README states "anything that speaks the contract can replace server/."

examples/gp_demo_ws.py (Python bridge)

- Run the benchmark scenario through the GP-enabled pipeline; at ~20 Hz emit
`tick { t, mean:[x,y]=position\_m[:2], cov = 2×2 block of NavigationOutput.covariance position block }`.
- Serve it over WebSocket (add a dev-only websockets dependency to the example, **not** to dvl_correction core) and point frontend/web/src/ws.ts WS_URL at it.
- Optionally honor play/pause/reset/set_path; minimally, stream the canned scenario.

What the demo shows

Toggle the GP on/off and watch the covariance ellipse: **without** the GP it balloons during a DVL dropout (and the mean drifts); **with** the GP it stays tight and the mean tracks truth — a vivid, intuitive picture of the benefit.

4. Tests (dvl_correction/tests/test_gp_velocity.py)

- **Unit:** predict returns finite (3,) mean + SPD (3,3) cov; variance at t = last+10s > variance at t = last+0.5s; is_outlier flags a ×5 spike and passes a normal sample; passthrough when not ready.
- **Integration:** a small pipeline with the GP enabled produces the same NavigationOutput shape as baseline, never raises, and gp_bridge_updates > 0 across a simulated gap.
- **Regression:** python3 -m unittest discover -s dvl_correction/tests stays green with the GP off

by default (existing tests untouched).

5. Reproduce

```
pip install -e ./dvl_correction[gp]
python examples/gp_dropout_benchmark.py          # prints RMSE table, writes plot
# optional live demo:
python examples/gp_demo_ws.py                    # serves ws://127.0.0.1:8080/ws
cd frontend/web && npm install && npm run dev      # open http://localhost:5173
```

Key takeaways

- Build a **synthetic ground-truth + dropout/outlier** scenario (reuse the test harness) and compare

position RMSE with/without the GP, focusing on dropout windows — the primary proof.

- The frontend needs **no changes**: emit the existing tick `{mean, cov}` contract from a small Python

WS bridge to visualize the covariance ellipse staying tight through a dropout.

- Lock it in with unit + integration tests; keep existing tests green (GP off by default).

Further reading

- 04 — Edge and SPDE roadmap

04 — Edge and SPDE Roadmap

What you'll learn: the staged path beyond the first build — (1) the $O(n)$ edge port of the velocity GP, and (2) the physics-informed ocean-current SPDE field — with scope, effort, and deps.

Prereqs: references/05, 06, 07; 02-gp-velocity-module-design.

Why it matters here: the eventual target is on-vehicle, and this is where the PDE-duality work lands.

Stage 0 (now) — GPyTorch velocity GP, demo-first

Covered by docs 00-03. Accuracy-focused, ground-side, torch-based. Deliberately not edge-deployable; everything below removes that limitation or extends capability.

Stage 1 — $O(n)$ edge port of the velocity GP

Goal: run the same DVL-velocity GP on the vehicle with no GPU/torch, at $O(n)$ cost.

How: swap the engine inside `GpDvlVelocityModel` for the **state-space / Kalman** form (references/05). A Matérn-3/2 temporal GP = a 2-state-per-axis linear SDE = a tiny Kalman filter/RTS smoother. The `predict()/ingest()` API and all pipeline wiring are **unchanged** (02 §7).

Reuse `imu-drift`: take the process-noise parameters from the crate's **Allan-variance** fit (`imu-drift/src/allan.rs`) and closed-form variances (`imu-drift/src/processes.rs`, e.g. `VelocityRandomWalk`, `GaussMarkov`) so the prior is grounded in the vehicle's real IMU characterization rather than learned hyperparameters (references/05 §5).

Hyperparameters: learn them once **off-board** with the Stage-0 GPyTorch model on logged data, then **freeze** them in the on-board recursion (references/08 §2f).

Effort: moderate. Pure numpy (or extend the `no_std` Rust crate). Validate by checking the state-space posterior matches the batch GP on the Stage-0 benchmark (they should agree to numerical precision for Matérn kernels), then re-run the RMSE comparison.

Deps: none new (drop torch). Optionally Rust if going fully embedded.

Stage 2 — Physics-informed ocean-current SPDE field

Goal: model the **current field** the vehicle moves through, to bound drift even without DVL bottom-lock, using the PDE duality.

How (the duality in action,

references/06-07):

1. Represent the current field as a Gaussian field defined by the **advection-diffusion SPDE**

$\partial X / \partial t + (\gamma \cdot \nabla) X - \nabla \cdot (D \nabla X) + \kappa^2 X = ?$ — γ = drift/current, D = diffusion (references/07 §2).

2. Discretize by **FEM** on a mesh of the operating area → a **GMRF** with **sparse precision** (references/06 §3); add **streamline-diffusion** stabilization where advection dominates.

3. Impose **boundary conditions** — Neumann "no-flow" at the seabed, Dirichlet at known inflows

(references/07 §3); optionally a

divergence-free kernel for incompressibility (references/07 §4).

4. Keep **Markov-in-time** (Kalman) over the GMRF for an efficient spatio-temporal filter (references/05 §7, references/08 §2c).

Use in navigation: the field provides a **current prior**; combined with water-relative velocity it reconstructs ground velocity during bottom-lock loss, bounding drift. It becomes a new measurement

source / process input to the EKF (a larger integration than Stage 0-1).

Effort: large. New subsystem (meshing, FEM assembly, sparse solves), a way to estimate γ/D (from data or an ocean model), and validation data. This is a research-grade extension, not a quick follow-up.

Deps: `scipy.sparse` (or a FEM library) for sparse assembly/solves; meshing (fmesher-style or a Python equivalent). Heavier than the rest; justified only when the current field is the priority.

Staging summary

Stage	Capability	Cost/where	New deps	Effort
0 (now)	velocity GP: denoise / adaptive R / bridge	$O(n^3)$, small n ; ground-side	<code>gpytorch</code> , <code>torch</code> (opt-in)	small
1	same GP, on-vehicle	$O(n^2)$; edge	none (drop <code>torch</code>)	moderate
2	physics-informed current field	$O(n) - O(n^{3/2})$; ground or edge	<code>scipy.sparse</code> / FEM / mesh	large

Recommended sequencing

1. Ship Stage 0; prove the benefit on the benchmark.
2. Do Stage 1 once the velocity GP earns its place — it's the real edge deliverable and reuses

`imu-drift`.

3. Treat Stage 2 as a separate research initiative when current-driven drift (not DVL dropout) becomes

the dominant error and survey data is available to fit/validate the field.

Key takeaways

- **Stage 1** = swap the velocity GP's engine to the **$O(n)$ state-space form** (same API, no pipeline

change), seed it from ``imu-drift`` Allan-variance params, freeze hyperparameters learned off-board → true edge deployment, no `torch`.

- **Stage 2** = the full **PDE-duality** payoff: an **advection-diffusion SPDE/GMRF** current field

with **boundary conditions**, bounding drift without bottom-lock — large, research-grade, separate.

- Sequence them; don't block the high-value velocity GP on the ambitious field model.

Further reading

- Back to references/06 and references/07 for the SPDE math.
- references/09 for all sources.